

## **WEEK 2 PROGRESS REPORT**

### **Foundation-Model Engineering and Prompting**

**Project: University Student-Support Case Agent**

**Course: BSE4104 – Emerging Trends in Software Engineering**

**Week: Week 2**

**Period: 7th September – 11th September 2026**

**Group: Group G**

### **1. Introduction**

During Week 2, the team focused on Foundation-Model Engineering and Prompting for the University Student-Support Case Agent. The main objective was to establish a working baseline interaction with a foundation model and develop a reliable prompt before introducing Retrieval-Augmented Generation (RAG), application tools, and agentic workflows in later weeks.

The team selected Google Gemini 3.7 Flash as the primary foundation model because it was accessible for development and prototyping through Google AI Studio. The implementation was done in Python using the google-genai library, with the API key stored securely in an environment file.

### **2. Activities Completed**

#### **2.1 Foundation Model Integration**

A basic Python application was created to communicate with the Gemini API. The initial implementation established the connection between the application and the foundation model and successfully generated responses to student-support questions.

The project environment was also configured with a Python virtual environment and the required dependencies:

- google-genai
- python-dotenv

The API key was stored in .env and excluded from version control using .gitignore.

#### **2.2 Model Behaviour Testing**

Initial experiments were conducted to understand how the foundation model behaves when application-specific information is unavailable.

For example, the model was asked about the status of CASE-1045 without being given any case information. Although Gemini correctly indicated that it did not have access to the university's internal database, it also generated plausible information about possible processing times, portals, and support procedures.

This demonstrated that a foundation model can produce a response that appears reasonable while still containing information that is not grounded in the application's actual data.

The same case was then tested with application information explicitly supplied in the prompt. When provided with the case status, department, and last updated date, the model correctly reported the supplied information. This demonstrated the importance of providing appropriate context to the model.

### **2.3 Prompt Specification Development**

Based on the initial experiments, Prompt Specification v1.0 was developed. It defined the assistant's role, task, information boundaries, missing-information behaviour, safety restrictions, and response style.

The prompt was then tested against different scenarios to identify weaknesses. Testing showed that the model could still recommend university-specific procedures that had not been provided by the application or an approved source.

The prompt was therefore refined into Prompt v1.1. The revised version strengthened information grounding, prohibited the use of general model knowledge to fill university-specific gaps, introduced explicit conflict handling, strengthened the assistant's scope, and prohibited unsupported recommendations about university offices, portals, departments, contact methods, and procedures.

### **2.4 Prompt Evaluation**

A set of 10 behavioural test cases was created to evaluate Prompt v1.1. The tests covered:

- Missing case information
- Valid case information
- Conflicting information
- Unknown university policies
- High-impact requests
- Unknown cases
- Known departments
- Out-of-scope questions
- Missing information fields
- Mixed available and unavailable information

The final evaluation produced a result of 10/10 tests passed.

This provided evidence that Prompt v1.1 established the intended baseline behaviour for the tested scenarios.

## **3. Challenges Encountered**

One challenge encountered during development was temporary API availability and quota limitations on the Gemini free tier. A 503 UNAVAILABLE error occurred during one request, which was resolved by retrying the request. A temporary 429 RESOURCE\_EXHAUSTED error was also encountered during evaluation before the affected test was successfully rerun.

These incidents highlighted the importance of considering model availability, request limits, and fallback options when selecting a foundation model. Gemini remains the primary model for the current Week 2 implementation, while a fallback model can be considered in later development.

Another important challenge was distinguishing between a model response that sounds correct and one that is actually grounded in approved application information. This became a major consideration in the prompt design and evaluation process.

#### **4. Deliverables Completed**

The following Week 2 outputs were completed:

1. Working Gemini foundation-model interaction in Python.
2. Model Selection Note.
3. Prompt Specification v1.0.
4. Prompt Version v1.1.
5. Prompt Version History documenting prompt changes and their rationale.
6. Ten-case prompt evaluation.
7. Week 2 project progress documentation.

#### **5. Learning Outcomes**

By the end of Week 2, the team established several important foundation-model engineering concepts. It was demonstrated that the model does not automatically have access to application-specific information and that prompting alone cannot provide missing data.

The team also established the distinction between different system components:

Prompting controls model behaviour, RAG will provide approved knowledge, tools will provide capabilities and actions, databases will store application state, and the agent workflow will coordinate these components.

The testing process also demonstrated that prompt engineering is an iterative engineering activity involving testing, observing failures, revising the prompt, and retesting rather than simply writing a single instruction.

#### **6. Next Steps**

In Week 3, development will progress from the manually supplied context used in Week 2 toward Retrieval-Augmented Generation (RAG). Approved university/course documents will be prepared as the knowledge base, and the system will retrieve relevant information before generating responses.

The next stage will focus on source grounding, retrieval quality, handling unsupported questions, and providing citations or source references where appropriate. This will establish the knowledge layer required before application tools and the bounded agent workflow are introduced in subsequent weeks.

## **7. Conclusion**

Week 2 successfully established the foundation-model baseline for the University Student-Support Case Agent. Gemini 3.7 Flash was integrated into the Python application, prompt specifications were developed and iteratively improved, and the final Prompt v1.1 achieved a 10/10 pass rate across the defined behavioural evaluation scenarios.

The work completed this week provides a tested foundation for the next stages of the project, particularly RAG, application tools, workflow state, memory, and the bounded agentic system.